

AI-BDM

Code Flow & Module Reference — every file, every function, in plain language.

This is the code-level companion to **AI-BDM_Architecture_And_Concepts_Guide.pdf** (which explains *what* the system does and the ideas behind it, for a reader with no technical background at all). This document goes one level deeper: it explains *how it's actually built* — every file, and inside each file, every important function — in plain language, without assuming the reader already codes in Python.

Every function description follows the same three-part pattern, like a recipe card:

- **What it needs** — the information this piece of code requires before it can start.
- **What it does** — the actual steps it takes, in plain English.
- **What it gives back** — the result it hands to whoever called it.

Read Section 1 first — the real story of what happens end to end. Section 3 is the detailed reference to come back to afterward, module by module, function by function.

Contents

- 1. The Complete Execution Trace — main.py, start to finish
- 2. The Two Entry Points (CLI vs. API) and the Async Job Layer
- 3. Module Reference — every file, every function, explained
 - 3.1 Core pipeline (Stages 1–3)
 - 3.2 Technology detection (Stage 5)
 - 3.3 Business intelligence / enrichment
 - 3.4 Reviews & Maps (Stage 2 harvesting + Stage 7)
 - 3.5 Evidence index / RAG (Stage 6)
 - 3.6 Storage & data rollup
 - 3.7 API & async job layer
 - 3.8 Supporting / standalone tools
- 4. Tests — how the codebase proves itself correct
- 5. How to Summarize This in One Minute

1. The Complete Execution Trace

This is exactly what happens, in order, when someone runs:

```
python main.py --query "find 3 dental clinics in Islamabad with no online booking"
```

Each line below is one function being called. The more a line is indented, the deeper it is — an indented line was called *by* the line directly above it, the way a phone call inside a phone call would be. This was traced directly from the real source code, not written from memory.

Entry point

main.py : main() — Reads the command you typed (--query, --concurrency, --no-rag, --no-maps) and hands off to run().

>> **main.py : run(...)** — The conductor of the whole process — every step below happens inside this one function, one after another.

Steps 1 & 2 — Understand the request, then search and scrape

>> **main.py : _plan_interactively(...)** — Wraps Steps 1+2 so that if the AI needs to ask you a clarifying question, it can do that in the same terminal window before continuing.

>> **phase1_pipeline.py : run_pipeline(...)** — The real engine for Steps 1 and 2 — everything below this line happens inside this one call.

>> **phase1_pipeline.py : moderate_user_query(...)** — A safety check that runs FIRST, before spending any money — refuses to continue if the request breaks usage policy.

>> **phase1_pipeline.py : deconstruct_intent(...)** — Figures out what you actually mean.

>> **LLM_planner.py : plan_query(...)** — Sends your sentence to the AI (Groq) with detailed instructions, and gets back a structured plan: industry, location, how many results, whether there's an extra condition to check, and whether it needs to ask you something first.

>> **model_router.py : select_model(...)** — Decides which specific AI model and settings this particular call should use.

>> **phase1_pipeline.py : discover_targets(...) / discover_places(...)** — Finds candidate business web addresses using Google Search and Google Maps (both via Serper.dev).

>> **phase1_pipeline.py : process_single_lead(...)** — Runs once for every candidate business found. This is the single biggest, busiest function in the whole project — everything below happens inside it, once per business.

>> **website_classifier.py : classify_homepage(...)** — A quick check: is this homepage worth fully reading, or does it look like a directory/junk page?

>> **route_filter.py : select_high_intent_routes(...)** — Out of every link on the site, picks just the handful worth actually reading (contact, about, services) instead of the whole site.

>> **discovery_classifier.py : classify_search_result(...)** — Tells apart a real single business from a directory page that should be mined for MORE businesses.

>> **relevance_scoring.py : score_relevance(...)** — A rule-based (no AI) check of how well this business actually matches what was asked for.

>> **tech_stack.py : analyze_raw_html(...)** — Figures out what software/technology this website runs on, using the page already downloaded — done for every business regardless of whether the request asked about technology.

>> **phase3/review_harvester.py : enrich_business(...)** — Looks for customer reviews on whichever platforms make sense for this kind of business.

>> **bi_providers.py : get_manager().discover_social_profiles(...)** — Finds the business's official social media pages.

>> **github_enrichment.py / youtube_enrichment.py** — Checks two platforms with genuine, rule-following official APIs.

>> **bi_providers.py : get_manager().discover_decision_makers(...)** — Tries several ways, in order, to find named people at the business: its own contact page first, then structured data on

the page, then LinkedIn (found via a normal Google search, never scraped directly), then a general search as a last resort.

>> **organization.py : build_organization(...)** — Groups whoever was found into a guessed org chart (departments, counts).

>> **business_intelligence.py : build_business_intelligence_summary(...)** — Combines everything found above into one tidy summary for this business.

>> **storage.py : commit_domain(...)** — Saves everything found for this business permanently — this is the actual "write it to disk" moment.

>> **storage_sync.py : sync_folder(...)** — Optional — also copies that folder to cloud storage (Cloudflare R2), if that's turned on.

Step 3 — Pick the right pages to focus on

>> **route_planner.py : plan_routes(...)** — Runs once for every business that was discovered.

>> **page_retrieval.py : hybrid_retrieve(...)** — Searches that one business's already-read pages for whichever ones are most relevant to the question — combining exact-word matching and meaning-based matching. This is the normal path and doesn't need to call the AI.

>> **LLM_planner.py (fallback call)** — Only used in the rare case the search above finds nothing good enough.

Step 4 — Answer the specific question (only if the request asked one)

>> **final_reasoning.py : answer_query(...)** — Runs once per business, but only when the request had an extra condition to check.

>> **(a Groq AI call, using only the real page text found above)** — Answers strictly from that real text, and says "not enough evidence" instead of guessing when the text doesn't actually answer the question.

Step 5 — Show the technology results (only if the request asked)

>> **tech_stack.py : get_stored_profile(...)** — Reads back the technology profile that was already worked out during Step 2 — nothing new is computed here unless something was somehow missed earlier.

Cleaning — build the one-row-per-business results file

>> **data_pipeline.py : run(...)** — Turns the detailed page-by-page records into one clean row per business.

>> **data_pipeline.py : to_business_level(...)** — The actual rollup logic — for each business, picks the best title/description and combines contact details found across all its pages.

Step 6 — Build a searchable memory of everything found (skip with --no-rag)

>> **rag/ingest_and_answer.py : run(...)** —

>> **rag/chunker.py : chunk(...)** — Breaks the text into small, manageable pieces.

>> **rag/embedder.py** — Turns each piece into a list of numbers representing its meaning (using a small, free, local AI model — no internet call).

>> **rag/store.py : get_store()** — Saves those number-lists into a local search database (Chroma).

>> **rag/retriever.py : retrieve(...)** — Searches that database for the pieces most relevant to this specific request.

Step 7 — Cross-check Google Maps (skip with --no-maps)

>> **phase3/google_maps.py : enrich(...)** —

>> **phase3/google_maps.py : find_place(...)** — Looks the business up on Google Maps and confirms the match — trying its address first, then its phone number, and only matching by name alone as a last resort.

Step 8 — Final quality check (always runs)

>> `accuracy_check.py : run(...)` — A read-only double-check over everything just saved — nothing is re-downloaded.

>> `accuracy_check.py : check_field_validity(...) / check_identity_consistency(...) / check_review_relevance(...)` — The three specific checks — are the phone/email/rating formatted correctly, does the business's name match across every source, and do the reviews actually mention this business.

2. The Two Entry Points (CLI vs. API) and the Async Job Layer

Everything in Section 1 can be reached two different ways: typed directly into a terminal, or requested by another computer program over the network.

main.py

The command-line way of running everything — this is what a person types directly.

`run(query, concurrency, no_rag, no_maps)`

What it needs: The plain-English request, how many pages to read at once, and two on/off switches for skipping Steps 6 and 7.

What it does: Runs Steps 1 through 8 shown in Section 1, in order, printing progress to the screen as it goes.

What it gives back: Nothing is returned directly — results are written to files (leads_clean.csv, leads_with_maps.csv, accuracy_report.txt).

`main()`

What it needs: Whatever was typed after "python main.py" on the command line.

What it does: Reads those typed options and calls `run()` with them.

What it gives back: Nothing — this is just the doorway into `run()`.

api.py

The network way of running things — lets another program (like the Laravel backend) trigger a research request instead of a person typing a command.

`run_pipeline_endpoint(req, response, idempotency_key)`

What it needs: A request with the plain-English query and how many pages to read at once, sent over the network.

What it does: Runs Steps 1+2 only (not the full 8 steps `main.py` runs) and waits until they're completely finished before replying. If the same request is sent twice with the same "Idempotency-Key" tag, the second call just repeats the first answer instead of doing the work again.

What it gives back: The same kind of results summary `main.py` prints, but as structured data instead of screen text.

`submit_job(req)`

What it needs: The same kind of request, but this time the caller doesn't want to wait.

What it does: Starts the work in the background immediately and hands back an ID for it right away.

What it gives back: A small confirmation: "job accepted, here's its ID, current status is queued."

`job_status(job_id)`

What it needs: The ID from `submit_job()`.

What it does: Looks up how that job is doing right now.

What it gives back: Its current stage, how far along it is, and a timestamp proving it's still alive.

`job_result(job_id, response)`

What it needs: The ID from `submit_job()`.

What it does: Checks whether that job has actually finished yet.

What it gives back: If it's still running: a polite "come back later." If it's done: the full results.

`cancel_job(job_id)`

What it needs: The ID from `submit_job()`.

What it does: Asks the job to stop at the next safe opportunity, keeping whatever it already found.

What it gives back: Whether the cancellation request was accepted (a job that already finished can't be cancelled, which isn't treated as an error).

`resume_job(job_id, req)`

What it needs: The ID of a job that already failed or was cancelled, plus a brand-new ID for the retry.

What it does: Starts a new job that searches for the exact same thing the original one was looking for. Businesses already found and saved by the original attempt are automatically skipped, so it isn't starting from zero.

What it gives back: A new job ID, running in the background just like `submit_job()`.

`job_runner.py`

The engine behind `submit_job()/job_status()/etc.` above — keeps track of every job's progress while it runs in the background.

`submit_job(req, resumed_from)`

What it needs: A structured request (and, only when called by `resume_job` below, a note of which earlier job this is a redo of).

What it does: Registers the job, then starts the actual pipeline work running in the background without making anyone wait for it.

What it gives back: Nothing directly — the job now exists and can be checked on.

`resume_job(original_job_id, new_job_id, new_correlation_id, tenant_ref)`

What it needs: The ID of an old job to restart, plus a new ID for the fresh attempt.

What it does: Copies the original request's search criteria onto a brand-new job and starts it.

What it gives back: Nothing directly — same as `submit_job()`, the new job now exists.

`get_status(job_id)`

What it needs: A job ID.

What it does: Looks up that job's current state.

What it gives back: Its stage, progress, and last-seen-alive timestamp.

`cancel_job(job_id)`

What it needs: A job ID.

What it does: Flags that job to stop as soon as it safely can.

What it gives back: True if the cancellation was accepted, False if the job had already finished.

`get_result(job_id)`

What it needs: A job ID.

What it does: Checks whether that job has reached a final state yet.

What it gives back: Nothing if it's still running; the complete results once it's done.

`job_translation.py`

A translator between two different ways of describing a search request.

`target_to_query(req)`

What it needs: A structured request (separate fields for industry, location, company size, etc.).

What it does: Converts those separate fields into the one plain-English sentence the rest of the system actually understands, since some of those fields (like company size) can't currently be honored and need to be flagged rather than silently ignored.

What it gives back: The resulting sentence, plus a list of any warnings about parts of the request that couldn't be fully honored.

`job_contracts.py`

Defines the exact, agreed-upon shape of every request/response the async job system uses, and sorts failures into understandable categories.

`classify_error(raw_message, correlation_id, exc)`

What it needs: A description of something that went wrong, and a tracking ID.

What it does: Looks at what kind of failure this actually was and sorts it into one of 8 standard categories (e.g. "bad input," "ran out of quota," "the website blocked us") instead of leaving it as an unstructured error message.

What it gives back: A structured error record a calling program can act on automatically (retry, alert someone, give up), rather than just a string of text.

3. Module Reference — Every File, Every Function

The rest of this document is a complete reference. Skip to whichever file you need — each one starts with why it exists, then a card for every important function in it.

3.1 Core Pipeline (Stages 1–3)

phase1_pipeline.py

The largest, most important file in the project. It's the real engine behind Steps 1 and 2 — everything about understanding a request, finding candidate businesses, reading their websites, and deciding which ones genuinely qualify happens here.

`run_pipeline(user_query, limit, concurrency, progress_cb, cancel_check)`

What it needs: The plain-English request, and a few optional settings (how many results, how many pages to read at once, and optional "check in"/"should I stop" callbacks for the async job system).

What it does: Runs the entire Step 1+2 process end to end: plans the request, searches for candidates, reads each candidate's website, and saves the ones that genuinely qualify.

What it gives back: One big results summary — how many were found, how many qualified, and the details of each.

`moderate_user_query(user_prompt)`

What it needs: The plain-English request.

What it does: Checks it against usage policy before anything else runs, so a problematic request never triggers any paid searches or scraping.

What it gives back: Whether the request is safe to proceed with, and why not if it isn't.

`deconstruct_intent(user_prompt)`

What it needs: The plain-English request.

What it does: Passes it to the AI planner (LLM_planner.py) and packages the structured plan that comes back.

What it gives back: The structured plan — industry, location, how many results, etc.

`discover_targets(session, query, limit, exclude_domains, start_page, per_page, max_pages, country_code, location)`

What it needs: A search phrase and various paging/limit settings.

What it does: Runs a Google web search (via Serper.dev) for candidate businesses, page by page.

What it gives back: A list of candidate businesses found.

`discover_places(session, query, limit, exclude_domains, country_code, location)`

What it needs: A search phrase.

What it does: Looks up real businesses directly through Google Maps (via Serper's Places data) — a faster, more reliable source of genuine businesses than a plain web search.

What it gives back: A list of real candidate businesses with map data attached.

`process_single_lead(session, semaphore, target, cache, exclude_keywords, include_keywords, industry, geo, country_code, phone_regex, user_query)`

What it needs: One candidate business, plus the plan's criteria (industry, location, keywords to require or exclude).

What it does: The single busiest function in the project — reads the business's website, decides whether it genuinely qualifies, harvests reviews and technology/social/decision-maker data, and saves everything if it qualifies. See Section 1 for the full breakdown of everything this calls.

What it gives back: A record describing what happened for this one business (qualified, rejected, or failed, with details).

`crawl_site(session, root_url, root_html, root_method)`

What it needs: A business's homepage address and its already-downloaded content.

What it does: Reads through the site's pages one at a time, keeping only what's actually needed in memory (never saving the raw HTML to disk).

What it gives back: The cleaned, readable text found across the site's pages.

`extract_contacts(html, text, country_code, phone_regex, site_domain, verify_mx, email_min_score)`

What it needs: One page's content.

What it does: Pulls out every email address and phone number it can find on that page, and optionally double-checks that an email's domain can actually receive mail.

What it gives back: The contact details found on that page.

`classify_relevance(industry, geo, name, page_text, domain, address, schema_organization)`

What it needs: A business's details and the industry/location being searched for.

What it does: A rule-based (non-AI) check of how well this business matches — the main relevance gate that filters out obvious non-matches before spending more effort.

What it gives back: A relevance verdict and score.

`qualify_lead(matched_exclude, matched_include, include_keywords, has_text)`

What it needs: Which required/excluded keywords were found across the business's pages.

What it does: Decides the final qualification outcome — genuinely qualified, or excluded because of a keyword the request said to avoid.

What it gives back: The qualification verdict.

`resolve_effective_limit(plan, limit)`

What it needs: The AI's planned result count, and an optional override.

What it does: Works out the real target number of results to aim for, and whether the search should be a thorough multi-round effort or a single quick pass.

What it gives back: The final target count and a yes/no for "search aggressively."

`load_cache() / load_last_run()`

What it needs: Nothing (reads from the local saved files).

What it does: Loads what's already been found and saved from previous runs, so already-known businesses aren't redone from scratch.

What it gives back: The previously-saved data.

`print_summary(summary)`

What it needs: The results summary from `run_pipeline()`.

What it does: Formats it into a readable block of text.

What it gives back: Nothing directly — prints straight to the screen.

LLM_planner.py

Every single AI (LLM) call related to planning a request lives in this one file — nothing else in the project talks to the AI for planning purposes.

`plan_query(user_query)`

What it needs: The plain-English request.

What it does: Sends it to the AI with detailed, carefully-written instructions covering exactly what to figure out: industry, location, how many results, whether there's an extra condition, and whether a clarifying question is needed first.

What it gives back: The structured plan as a dictionary of fields.

`moderate_query(user_query)`

What it needs: The plain-English request.

What it does: Asks the AI to judge whether the request is safe/allowed, before anything else happens.

What it gives back: Whether it's safe, and if not, why.

`classify_business(industry, geo, name, page_text)`

What it needs: A business's scraped page text and the industry/location being searched for.

What it does: Asks the AI to judge whether this business genuinely matches — used as a careful double-check for borderline cases.

What it gives back: A relevance classification.

`generate_query_variations(user_query, industry, geo, exclude, max_variations)`

What it needs: The original request and a list of search phrases already tried.

What it does: Asks the AI for a handful of alternative ways to search for the exact same thing (never a different industry or location) — used only when the original search has run dry but more results are still needed.

What it gives back: A list of new search phrases to try.

`call_llm(client, messages, response_format)`

What it needs: The prepared AI conversation and the expected reply format.

What it does: The shared low-level function that actually sends any request to the AI provider (Groq), through the central model-routing rules.

What it gives back: The AI's raw reply text.

`get_client()`

What it needs: Nothing (reads the API key from the environment).

What it does: Creates the connection to the Groq AI service.

What it gives back: A ready-to-use client object.

model_router.py

The single place that decides which specific AI model gets used for which kind of task — nowhere else in the project is allowed to hardcode a model name.

`select_model(task)`

What it needs: A label describing what kind of task this AI call is for (e.g. "quick classification" vs. "deep reasoning").

What it does: Looks up the right model and settings for that specific kind of task — cheaper/faster models for simple decisions, a stronger model for genuine reasoning, with an automatic backup model if the first choice fails.

What it gives back: Which model to use and its settings.

`estimate_cost(model, prompt_tokens, completion_tokens)`

What it needs: Which model was used and roughly how much text went in and came out.

What it does: Works out a rough dollar cost for that one AI call, purely for logging/monitoring purposes.

What it gives back: An estimated cost in dollars.

relevance_scoring.py

A deterministic (non-AI, rule-based) way of scoring how well a business matches a request — used as the fast, cheap first-pass filter before anything more expensive happens.

`score_relevance(industry, geo, name, domain, sample_text, address, schema_organization, skip_safety_net)`

What it needs: A business's details and the industry/location being searched for.

What it does: Compares them using straightforward rules (does the text mention the right industry, does the address match the requested area) rather than asking the AI — much faster and free.

What it gives back: A relevance score and verdict (a strong match, a possible match, or a rejection).

discovery_classifier.py

Sorts every search result into one of three buckets: a real single business, a directory/listing page worth mining for more businesses, or noise to ignore.

`classify_search_result(url, title)`

What it needs: One search result's web address and title.

What it does: Applies a set of rules to decide which of the three buckets it belongs in.

What it gives back: The classification, with a confidence level.

`extract_businesses_from_html(html, source_url, source_name, confidence, seen_hosts, needed)`

What it needs: A directory-style page's content.

What it does: Pulls out links that look like they point to individual real businesses, skipping ones already seen.

What it gives back: A list of newly-found candidate businesses.

`extract_businesses_from_source(session, source_url, source_name, confidence, seen_hosts, needed)`

What it needs: A directory page's web address.

What it does: Downloads that page and then runs `extract_businesses_from_html()` on it.

What it gives back: The same list of newly-found candidates.

`is_utility_link(host, path, anchor)`

What it needs: One link found on a page.

What it does: Checks whether it's just a sign-up/login/help/advertising link rather than a real business page.

What it gives back: True or false.

`domain_key(url_or_host) / compute_content_hash(html) / confidence_rank(confidence)`

What it needs: A web address, a page's content, or a confidence label.

What it does: Three small helper checks: normalizing a domain name, creating a fingerprint of a page's content (to detect duplicates), and turning a confidence label into a sortable number.

What it gives back: The normalized domain, the fingerprint, or the sortable number, respectively.

website_classifier.py

Decides, cheaply and quickly, whether a homepage is worth a full, expensive crawl at all.

`classify_homepage(domain, url, html, tech_raw, user_query)`

What it needs: A homepage's already-downloaded content.

What it does: Combines quick rule-based signals with, when needed, a fast AI check to decide: deep-crawl this site, treat it as a directory to mine, or skip it entirely.

What it gives back: The decision, with the reasoning behind it.

`mine_homepage_businesses(html, url, domain, classification, needed)`

What it needs: A homepage already classified as directory-like.

What it does: Extracts links to the real individual businesses listed on it.

What it gives back: A list of newly-found candidate businesses.

`extract_homepage_signals(html, url, domain, tech_raw) / rule_based_score(signals)`

What it needs: A homepage's content.

What it does: Pulls out cheap, obvious signals (like how many outbound links it has) and turns them into a "how directory-like is this" score, with no AI involved.

What it gives back: The signals, and the score.

`is_non_english(html_lang, text_blob)`

What it needs: A page's declared language and its actual text.

What it does: Makes a best-effort guess at whether the page is confidently NOT in English.

What it gives back: True or false (defaults to "assume English" when unsure).

`route_filter.py`

Given a business's homepage, works out which of its own links are actually worth reading (contact, about, services) instead of the whole site.

`select_high_intent_routes(homepage_html, base_url, user_query, knowledge_gaps)`

What it needs: A homepage's content and address.

What it does: Runs the full pipeline of extracting links, filtering out junk, scoring what's left by how likely it is to contain useful information, and picking the best few.

What it gives back: The chosen pages to actually read, each with a priority and a reason.

`select_routes_for_site(website, user_query, knowledge_gaps)`

What it needs: A business's web address.

What it does: The same process as above, but starting from what's already been saved for that site rather than fresh HTML.

What it gives back: The chosen pages.

`extract_links(homepage_html) / normalize_urls(hrefs, base_url) / filter_junk(urls, base_url, config)`

What it needs: A homepage's raw links.

What it does: Three cleanup steps in sequence: pull out every link, turn them into full, consistent web addresses, then throw out ones that are obviously not useful content (like login pages or social share buttons).

What it gives back: The cleaned, de-duplicated, junk-free list of links.

`score_url(url) / rank_urls(urls) / score_candidate(link)`

What it needs: A link (or list of links).

What it does: Scores each one by how likely its web address is to lead to useful information (e.g. a URL containing "contact" or "about" scores higher), with no AI involved.

What it gives back: A score, or the list sorted best-to-worst.

`classify_link(link)`

What it needs: One candidate link with its surrounding context.

What it does: Decides how confident to be that this link is actually useful, based only on where it sits in the page's navigation.

What it gives back: A confidence level and the reason for it.

`route_planner.py`

Stage 3's actual entry point — for one already-crawled business, decides which of its saved pages are most relevant to answer a specific question.

`plan_routes(website, query)`

What it needs: A business's web address and the question being asked.

What it does: Combines a fast rule-based shortlist with the search-first retrieval in `page_retrieval.py`, only falling back to an AI call in the rare case nothing useful turns up automatically.

What it gives back: The selected pages, with a confidence level for the choice.

`load_page_metadata(domain) / build_link_graph(domain, pages)`

What it needs: A business's domain.

What it does: Reads back everything already saved about that business's pages and how they link to each other.

What it gives back: That saved data, ready to be scored.

`rule_based_prefilter(pages, graph) / heuristic_plan(candidates)`

What it needs: A business's saved pages.

What it does: Throws out obviously irrelevant pages and ranks what's left using simple rules — the deterministic fallback used when nothing fancier is needed.

What it gives back: The pre-filtered/ranked shortlist.

page_retrieval.py

The actual "search this business's own pages for the right answer" engine that Stage 3 relies on in the normal case — no AI call needed.

`hybrid_retrieve(domain, query, top_k)`

What it needs: A business's domain and the question being asked.

What it does: Searches that business's already-read pages using both exact keyword matching and meaning-based matching together, since combining the two finds more of what's actually relevant than either alone.

What it gives back: The best-matching page excerpts, ranked.

`build_index(domain)`

What it needs: A business's domain.

What it does: Makes sure that business's pages are ready to be searched quickly (builds the local search index if it doesn't already exist).

What it gives back: Whether the index is ready.

`rerank(hits, query, top_n)`

What it needs: A first-pass list of matches.

What it does: Re-orders them using a combined, weighted scoring rule — no AI involved, just consistent math.

What it gives back: The final, re-ordered shortlist.

page_patterns.py & page_intelligence.py

Two small, no-AI helper files: one spots repeating link patterns on a homepage, the other builds a reusable, structured summary of any single page.

`page_patterns.py : extract_homepage_candidates(html, base_url, root_domain) / detect_patterns(candidates)`

What it needs: A homepage's links.

What it does: Groups links that share a common web-address pattern (e.g. every product page under /shop/...) so repeating structures are recognized.

What it gives back: The grouped pattern summaries.

`page_intelligence.py : build_page_context() / classify_page_type(...) / extract_structure(...) / extract_content(...)`

What it needs: One page's cleaned text and headings.

What it does: Builds a consistent, structured summary of that page — what type of page it is (contact/about/services/etc.), its headings, and short excerpts of its real content.

What it gives back: That structured per-page summary, reused throughout the project.

`page_intelligence.py : enrich_anchors(domain)`

What it needs: A business's domain.

What it does: Goes back and fills in link-text information for every page already saved for that business.

What it gives back: Nothing directly — updates the saved data.

`final_reasoning.py`

Stage 4 — the one place in the whole project that actually writes a synthesized answer to a specific question, always grounded in real evidence (the RAG pattern).

`answer_query(domain, query, top_n)`

What it needs: A business's domain and the specific question being asked.

What it does: Retrieves the most relevant already-read pages (via `page_retrieval.py`), then asks the AI to answer strictly from that real text, citing exactly which pages it used — and to say "not enough evidence" rather than guess when the text doesn't actually answer the question.

What it gives back: The answer, its confidence level, and which pages it was based on.

`domain_utils.py & time_utils.py`

Two tiny, dependency-free helper files used everywhere in the project — kept deliberately small so importing them never accidentally drags in the heavy AI libraries.

`domain_utils.py : domain_key(url_or_host)`

What it needs: Any web address or hostname.

What it does: Strips it down to its clean, registered domain (e.g. turns "help.example.com" into "example.com").

What it gives back: The clean domain name — the single, consistent way this project identifies a business.

`domain_utils.py : strip_tracking(url)`

What it needs: A web address.

What it does: Removes tracking parameters (like `utm_source`, `gclid`) so the same real page isn't treated as different addresses.

What it gives back: The cleaned web address.

`time_utils.py : utc_now_iso(timespec)`

What it needs: Nothing (reads the current time).

What it does: Produces the current time in one single, consistent written format used everywhere a timestamp is saved.

What it gives back: The current time as text, e.g. "2026-08-06T12:00:00Z".

`prompt_cache.py`

A small cache so the exact same question is never sent to the AI twice.

`get(task, model, messages) / set(task, model, messages, response)`

What it needs: Which task/model/question was asked (get), or the answer to save (set).

What it does: Checks whether this exact question has already been asked and answered before reusing it; otherwise saves a fresh answer for next time.

What it gives back: The previously-saved answer, if one exists (get); nothing directly (set).

3.2 Technology Detection (Stage 5)

tech_stack.py

Everything related to figuring out what software/technology a business's website runs on — CMS, booking tools, analytics, hosting, and more.

`analyze_raw_html(domain, html, headers, cookies)`

What it needs: A business's homepage content, already downloaded — no extra fetch needed.

What it does: Runs 4 separate detection methods (a legacy fingerprint engine, an extended engine covering cookies/page-structure/scripts, a domain-record lookup, and a robots.txt check) and merges everything they find into one profile.

What it gives back: The combined raw technology profile for that business.

`build_website_profile(domain, raw)`

What it needs: The raw detection results from `analyze_raw_html()`.

What it does: Organizes the raw findings into readable categories and business-friendly groupings (e.g. "booking system," "analytics," "CMS").

What it gives back: The full, organized technology profile.

`get_stored_profile(domain, allow_fallback_scan)`

What it needs: A business's domain.

What it does: The normal way anything else in the project reads a technology profile — always checks what's already saved first, and only re-scans as a rare fallback.

What it gives back: The saved technology profile.

`build_normalized_tech_stack(raw) / build_categories_view(raw) / build_versions_view(raw)`

What it needs: The raw detection results.

What it does: Three different ways of re-organizing the same raw findings for different purposes (a curated summary, a by-category view, a by-version view).

What it gives back: Each respective organized view.

`generate_sales_signals(domain) / generate_sales_signals_from_profile(profile)`

What it needs: A business's domain, or an already-built profile.

What it does: Looks for patterns in the technology profile that suggest a sales opportunity (e.g. an outdated platform, no booking system).

What it gives back: A list of sales signals with confidence levels.

`has_crm(domain) / has_login_page(domain) / has_customer_portal(domain) / is_wordpress(domain) / is_shopify(domain) / is_outdated_stack(domain) / (and similar yes/no checks)`

What it needs: A business's domain.

What it does: Each is a simple, specific yes/no question answered by looking at the saved technology profile.

What it gives back: True or false.

3.3 Business Intelligence / Enrichment (part of Stage 2)

bi_providers.py

The coordinator that ties every enrichment source below together behind one consistent interface, so the rest of the project doesn't need to know which specific source found what.

`get_manager()`

What it needs: Nothing (reads which providers are enabled from configuration).

What it does: Builds the manager object that knows how to call every configured enrichment source in the right order.

What it gives back: The ready-to-use manager.

`make_result(provider)`

What it needs: Which provider produced a result.

What it does: Wraps that provider's findings into the one standard response shape every enrichment source uses, so results from different sources can be handled identically.

What it gives back: The standardized result.

social_discovery.py

Finds a business's own official social media links, directly from its website — never guesses or searches externally for this part.

`extract_social_links(external_links)`

What it needs: Every outbound link found while reading a business's website.

What it does: Filters that list down to ones that are genuinely the business's own social media profiles.

What it gives back: The confirmed social profile links, organized by platform.

linkedin_discovery.py

Finds a business's LinkedIn company page and any named decision-makers on LinkedIn — always through a normal Google search (never by directly scraping LinkedIn, which its terms don't allow).

`discover_company_linkedin(session, company_name, domain, city, country, industry)`

What it needs: A business's name, location, and industry.

What it does: Tries several carefully-built Google search phrases in order until one confidently finds the business's real LinkedIn company page.

What it gives back: The matched LinkedIn page, with a confidence score.

`discover_decision_makers(session, company_name, domain, city, country, industry, max_people)`

What it needs: The same business details.

What it does: Searches for LinkedIn profile pages of people who plausibly work there, using targeted search phrases built around common executive job titles.

What it gives back: A list of candidate people, each with a confidence score.

`rank_linkedin_candidates()`

What it needs: A list of possible LinkedIn matches.

What it does: Scores each one for how likely it genuinely is the right page, so the best match can be picked with confidence.

What it gives back: The candidates, ranked.

`generate_google_xray_queries(...)` / `generate_decision_maker_xray_queries(...)`

What it needs: A business's name/location/industry, or the specific executive titles being searched for.

What it does: Builds the actual search phrases to try — this technique of searching a specific site through a normal search engine is called an "X-Ray search."

What it gives back: A list of search phrases to try in order.

`is_cache_fresh(data)` / `save_linkedin_candidates(store, domain, company_page, profiles)`

What it needs: Previously-saved LinkedIn data.

What it does: Checks whether saved data is recent enough to trust, or saves freshly-found data for next time.

What it gives back: True/false, or nothing (just saves).

linkedin_enrichment.py & linkedin_finder.py

Turn a confirmed LinkedIn match into more detailed, structured company data.

`linkedin_enrichment.py` : `enrich_linkedin_company(match, website_domain, provider)`

What it needs: An already-confirmed LinkedIn match.

What it does: Builds a fuller structured record about that company from LinkedIn, without ever failing the whole process if this one step doesn't work.

What it gives back: The enriched LinkedIn company record.

`linkedin_finder.py` : `find_linkedin(session, name, website, geo)`

What it needs: A business's name, website, and location.

What it does: An alternative, simpler LinkedIn-matching entry point.

What it gives back: The matched LinkedIn page and confidence.

github_enrichment.py & youtube_enrichment.py

The two enrichment sources that use genuine, fully rule-following official APIs (not search-based guessing).

`github_enrichment.py` : `enrich_github(session, github_url)`

What it needs: A business's GitHub page address, if one was found.

What it does: Fetches its public profile information directly from GitHub's own official API.

What it gives back: The public GitHub profile data.

`youtube_enrichment.py` : `enrich_youtube(session, youtube_url)`

What it needs: A business's YouTube channel address, if one was found.

What it does: Fetches its public channel information directly from YouTube's own official API.

What it gives back: The public channel data (subscriber count, etc.).

decision_maker_extractor.py, schema_org_extractor.py & public_search_decision_makers.py

Three different ways of finding named people at a business, tried in priority order (most reliable first).

`decision_maker_extractor.py` : `extract_decision_makers(page_url, text, company, external_links_on_page)`

What it needs: An About/Team/Leadership page's text.

What it does: Source #1 (highest priority): scans the business's own page for named people and their job titles.

What it gives back: The people found, with their names, titles, and department guesses.

decision_maker_extractor.py : extract_contact_page_people(page_url, text, external_links_on_page)

What it needs: A Contact/Support/Locations page's text.

What it does: A similar scan, specifically for contact-style pages.

What it gives back: The people found there.

schema_org_extractor.py : extract_schema_org(html_or_soup, page_url)

What it needs: A page's raw content.

What it does: Source #2: reads structured data many websites embed for search engines (called Schema.org/JSON-LD data), which sometimes directly lists staff.

What it gives back: Any people found in that structured data.

public_search_decision_makers.py : discover_via_public_search(session, domain)

What it needs: A business's domain.

What it does: Source #4, the last resort: tries general web searches scoped to that specific domain, looking for named people mentioned anywhere.

What it gives back: Any people found this way.

organization.py & buying_committee.py

Take whichever people were found above and make sense of them as an organization.

organization.py : build_organization(people)

What it needs: The list of people found by any of the sources above.

What it does: Groups them into a guessed departmental structure.

What it gives back: A breakdown by department, with counts.

**buying_committee.py : infer_likely_ownership(person) /
annotate_buying_committee(people)**

What it needs: One person's details, or the whole list.

What it does: Makes an educated guess at which business decisions that person is likely to have a say in, based on their title.

What it gives back: The same person record, with a "likely_owner_of" field added.

business_intelligence.py

Rolls every enrichment source above into one final, consolidated per-business summary.

build_business_intelligence_summary(domain, social_profiles, linkedin_company, decision_makers, organization, github_profile, youtube_profile)

What it needs: The separate results from every enrichment source above, for one business.

What it does: Combines them all into a single, tidy summary document for that business.

What it gives back: The consolidated business-intelligence summary.

email_extractor.py

Focused specifically on finding and validating email addresses from scraped pages.

domain_has_mx(domain, timeout)

What it needs: A domain name found in an email address.

What it does: Checks whether that domain is actually set up to receive email at all (a technical mail-server check), catching obviously fake or broken addresses.

What it gives back: True, False, or "couldn't determine" if the check itself failed.

3.4 Reviews & Maps (Stage 2 Harvesting + Stage 7)

phase3/review_harvester.py

Finds and collects genuine customer reviews for a business, from whichever platforms actually make sense for its type of business.

enrich_business(domain, name, industry, geo, page_title, address, phone)

What it needs: A business's details.

What it does: Works out which review platforms are relevant for this kind of business (a dental clinic and a marina check completely different sites), then checks each one.

What it gives back: Reviews found, organized by platform.

enrich_platform(session, domain, name, address, geo, platform, hosts, phone)

What it needs: One specific review platform to check.

What it does: Searches for and confirms the business's listing on that one platform, then pulls whatever review text is genuinely, publicly available.

What it gives back: That platform's result — matched or not, with any reviews found.

extract_reviews(html)

What it needs: A review page's content.

What it does: Pulls out only review fields that are explicitly, publicly shown — never guesses or invents review content.

What it gives back: The extracted reviews.

platforms_for_category(category)

What it needs: A business category (e.g. "dental," "marina").

What it does: Looks up which review platforms are actually relevant to check for that category.

What it gives back: The list of relevant platforms.

phase3/business_category.py

Sorts a business into a category so review_harvester.py above knows which platforms to check.

categorize_domain(domain, industry, company_name, page_title)

What it needs: A business's details.

What it does: The normal, cache-first way of getting a business's category — checks what's already saved before working it out fresh.

What it gives back: The business's category.

categorize(industry, company_name, page_title)

What it needs: The same details.

What it does: The actual rule-based matching logic.

What it gives back: The best-matching category, or "uncategorized" if nothing fits.

phase3/google_maps.py

Stage 7 — confirms each business against its real Google Maps listing.

enrich(in_path, out_path, geo)

What it needs: The file of cleaned business results, and the general search location.

What it does: Runs the Maps check for every business in that file.

What it gives back: A new file with rating/review-count/match-confidence columns added.

`find_place(session, name, address, geo, phone)`

What it needs: One business's name, address, location, and phone number.

What it does: Looks it up through Google Maps and tries to confirm the match — first by comparing the address, then the phone number, and only by name alone as a last resort when there's exactly one candidate.

What it gives back: The matched listing (or no match), the confidence, and which method confirmed it.

`enrich_domain(session, domain, name, address, geo, phone, max_age_days)`

What it needs: A single already-saved business.

What it does: The cache-first version of `find_place()` for one specific business — reuses a recent saved result instead of re-checking every time.

What it gives back: The Maps match result for that business.

`phase3/store.py`

Phase 3's own small, simple save/load system for review and Maps data — separate from the main `storage.py`.

`load(domain, platform, max_age_days) / save(domain, platform, record)`

What it needs: A business's domain and which platform's data to load or save.

What it does: Reads back a previously-saved record (only if it's recent enough), or writes a fresh one.

What it gives back: The saved record, or nothing on save.

3.5 Evidence Index / RAG (Stage 6)

rag/ingest_and_answer.py

The main entry point for Stage 6 — what actually gets called by main.py.

run(query, discovered_domains, leads_json_path, k)

What it needs: The request and which businesses were found.

What it does: Coordinates the whole Stage 6 process: breaking text into pieces, turning them into meaning-vectors, saving them, and searching them for this specific request.

What it gives back: A combined answer plus a per-business evidence summary, also printed to the screen.

rag/chunker.py, rag/embedder.py & rag/store.py

The three building blocks of the searchable evidence index, in order.

rag/chunker.py : chunk(doc)

What it needs: One document's full text (a scraped page, or a review).

What it does: Breaks it into smaller, manageable pieces small enough for the meaning-search model to handle well, keeping a bit of context (like the page's title) with each piece.

What it gives back: A list of text chunks.

rag/embedder.py (embedding function)

What it needs: A text chunk.

What it does: Runs it through the small, local, free AI model that turns text into a list of numbers representing its meaning.

What it gives back: That list of numbers (the "embedding").

rag/store.py : get_store()

What it needs: Nothing (reads which storage backend is configured).

What it does: Connects to the local search database (Chroma) where all the embeddings are kept.

What it gives back: A ready-to-use connection to that database.

rag/retriever.py

The actual search step — the "R" in RAG.

retrieve(question, store, embedder, business, k)

What it needs: A question, the search database, and which business to search within.

What it does: Turns the question into its own meaning-vector, then finds the stored chunks whose meaning is closest to it.

What it gives back: The most relevant chunks found, ready to be handed to Stage 4 for an actual answer.

rag/ingest_from_storage.py, rag/ingest_from_csv.py & rag/ingest_reviews.py

Three different starting points for feeding real data into the search index, depending on where the data is coming from.

rag/ingest_from_storage.py : iter_source_docs_from_high_intent(...) / iter_source_docs_from_storage(...)

What it needs: Nothing extra — reads directly from the storage/ folders.

What it does: Walks through everything already saved for a set of businesses and prepares it to be chunked and embedded.

What it gives back: A stream of documents ready for rag/chunker.py.

A few other files in rag/ (generator.py, ask.py, eval.py, top_matches.py, match_query.py, llm_match_query.py) are standalone diagnostic/experimentation tools — useful for a developer manually testing the search index, but confirmed **not** called by the real

pipeline (rag/ingest_and_answer.py above never imports them). Worth knowing they exist, but they're not part of what actually runs when a request comes in.

3.6 Storage & Data Rollup

storage.py

The main save/load system — everything about writing a business's results to disk and reading them back goes through here.

`get_store()`

What it needs: Nothing.

What it does: Gives back the one shared connection to the local storage system, used everywhere the project needs to save or read business data.

What it gives back: That shared storage object, which itself provides `commit_domain()` (save everything found for one business) and various read functions.

storage_sync.py

An optional add-on that copies everything storage.py saves locally out to cloud storage (Cloudflare R2) as well, for backup.

`get_provider()`

What it needs: Nothing (reads whether remote sync is configured).

What it does: Checks whether cloud sync is turned on and set up correctly.

What it gives back: The ready-to-use cloud sync connection, or nothing if it's not configured — never an error either way.

data_pipeline.py

Turns the detailed, page-by-page saved records into the clean, one-row-per-business results files people actually look at.

`run(path, dry_run, scope_all, routes, tech_stacks)`

What it needs: Which saved records to process.

What it does: Runs the full cleaning process end to end: load, clean, roll up to one row per business, check for problems, and write the final files.

What it gives back: Nothing directly — writes `leads_clean.csv`, `leads_clean.json`, a quarantine file for anything dropped, and a data-quality report.

`to_business_level(pages)`

What it needs: The detailed, per-page records.

What it does: Combines every page belonging to the same business into a single, best-effort business record — picking the best title, combining contact details found across pages.

What it gives back: One record per business.

`clean_rows(rows, expected_industry, expected_geo) / dedupe_pages(rows)`

What it needs: The raw saved records.

What it does: Normalizes messy fields and removes duplicate/noisy rows.

What it gives back: The cleaned, de-duplicated rows, plus a separate list of what was rejected and why.

`govern(businesses)`

What it needs: The rolled-up business records.

What it does: Checks each one against a set of validity rules.

What it gives back: Which ones passed, and which failed with a reason.

`write_clean_export(businesses, path) / write_clean_export_json(...) / write_quarantine(...) / write_report(...)`

What it needs: The final processed data.

What it does: Writes it out to the actual result files people open — a spreadsheet, a JSON version, a "what got dropped and why" file, and a readable summary report.

What it gives back: Nothing directly — these are the files that end up on disk.

accuracy_check.py

Stage 8 — the final, read-only quality check over everything the pipeline just produced.

`run(geo, path)`

What it needs: Which results file to check.

What it does: Runs all three checks below over every row and writes a combined report.

What it gives back: Nothing directly — writes `accuracy_report.txt`.

`check_field_validity(row)`

What it needs: One business's row of results.

What it does: Checks whether its phone number, email, address, and rating are validly formatted.

What it gives back: A list of anything wrong.

`check_identity_consistency(domain, company_name, page_title)`

What it needs: A business's details from different sources.

What it does: Checks whether its name is consistent across its own site, Google Maps, and review platforms — catching a case where the data actually belongs to a different, similarly-named business.

What it gives back: Whether the identity is consistent, and a warning if not.

`check_review_relevance(domain, company_name, geo)`

What it needs: A business's harvested reviews.

What it does: Checks whether reviews that came from a general search (like Reddit) actually mention this specific business, rather than something unrelated.

What it gives back: A list of any reviews that look off-topic.

3.7 API & Async Job Layer

Already covered in full, function by function, in Section 2 above (api.py, job_runner.py, job_translation.py, job_contracts.py) — not repeated here to avoid duplication.

3.8 Supporting / Standalone Tools

scripts/patch_wappalyzer.py

A one-time maintenance tool, not run during normal operation.

`main()` / `verify()`

What it needs: Nothing (operates on the vendored technology-detection package).

What it does: Used only when updating the technology-detection package to a newer version in the future, to re-apply the same fixes that were needed last time and confirm they worked.

What it gives back: A pass/fail confirmation.

async_scraper/ is a separate, self-contained scraping toolkit that lives in the same repository but is **not called by main.py's pipeline at all** — an independent utility, not part of the production call chain described in Section 1.

vendor/wappalyzer/ is a working copy of a third-party technology-detection package, committed directly into this project (rather than installed normally) because the publicly published version was found to silently change over time. See the Architecture guide, Section 8, for the full story.

4. Tests — How the Codebase Proves Itself Correct

115+ automated tests, organized by how much of the real system each one touches:

- **tests/smoke/** — do the core pieces even load correctly, and does the final quality check work against sample data? No network, no API keys, runs in seconds.
- **tests/unit/** — pure logic checked in isolation (scoring rules, path handling) with no real files or network involved at all.
- **tests/mockbed/** — real logic, but with the network/AI calls replaced by fakes, so the actual retry/caching/job behavior can be proven without needing real credentials or spending real money.
- **tests/contract/** — checks that example requests/results actually match the agreed-upon shapes exactly.
- **tests/integration/** — the only tests that touch real, paid services (Groq, Serper, ScrapingBee, Apify) — deliberately kept out of the everyday test run, only used when specifically asked for.

Full detail — 142 hand-written test cases mapped to exactly what's already automated — is in AI-BDM_QA_Test_Plan.xlsx.

5. How to Summarize This in One Minute

If your manager asks "what did you actually build," here's the short version:

"I built the Python research engine for a B2B lead-generation system. It takes one plain-English request, uses an AI model to understand it, searches the web and reads real business websites, cross-checks each business against Google Maps and its own customer reviews, answers any specific question the request asked using real evidence instead of guessing, and runs its own automated quality check before handing back results. It's exposed both as a command-line tool and as a web API — including a version that runs in the background with progress tracking, cancellation, and the ability to resume — so a separate backend system can call it as part of a larger product. The whole thing has 115+ automated tests, is fully documented for handover, and runs in Docker alongside the rest of the system."

If asked to go one level deeper on any single piece, Section 3 gives you the exact file and function to point to, and what it does in plain language.